Relational Databases

Contents

Data Collections

Data Collections I

- Data collections are probably as old as science
- Collecting data, evidence and so on was already done in ancient greece and rome

Data Collections II

F17 2011

	A	B	C	D	E	F
1		Filiale	Standort	Land	Umsatz	Jahr
2		1	München	Deutschland	150,000 €	2010
3		2	Berlin	Deutschland	250,000 €	2010
4		3	Hamburg	Deutschland	85,000 €	2010
5		4	Prag	Tschechien	180,000 €	2010
6		5	Pilsen	Tschechien	40,000 €	2010
7		6	Salzburg	Österreich	45,000 €	2010
8		9	Wien	Österreich	40,000 €	2010
9		1	München	Deutschland	183,000 €	2011
10		2	Berlin	Deutschland	260,000 €	2011
11		3	Hamburg	Deutschland	70,000 €	2011
12		4	Prag	Tschechien	220,000 €	2011
13		5	Pilsen	Tschechien	90,000 €	2011
14		6	Salzburg	Österreich	60,000 €	2011
15		7	Innsbruck	Österreich	50,000 €	2011
16		8	Budweis	Tschechien	8,000 €	2011
17		9	Wien	Österreich	110,000 €	2011
18						

- Nowadays we work with applications like Excel or Word to collect data
- Can be relatively useful if you are only interested in the data

Problems I

- **Redundancy:** means we have multiple entries which contain the same information in our data collection
- **Inconsistency:** means that information that refers to a specific topic contradicts each other
- **Merging Data:** for example fusioning two excel sheets without overlap can either be very difficult or can’t be done at all
- **Expanding the Collection:** when you want to add new data to your collection multiple questions like Where to put this expansion? / How to save the expansion? / Do I have to change my tables? arise

Problems II

- **Multiple Users:** often aren't able to simultaneously generate new entries in a data collection
- **Loss of data:** If data is lost, the correlations in the data system often can't be reproduced
- **Safety concerns:** we can only grant access to the whole file or not but sometimes we want some users to be able to work only with specific parts of the data

Basics

Definition I

„A method for structuring data in the form of sets of records or tuples so that relations between different entities and attributes can be used for data access and transformation.“

~Burroughs, 1986

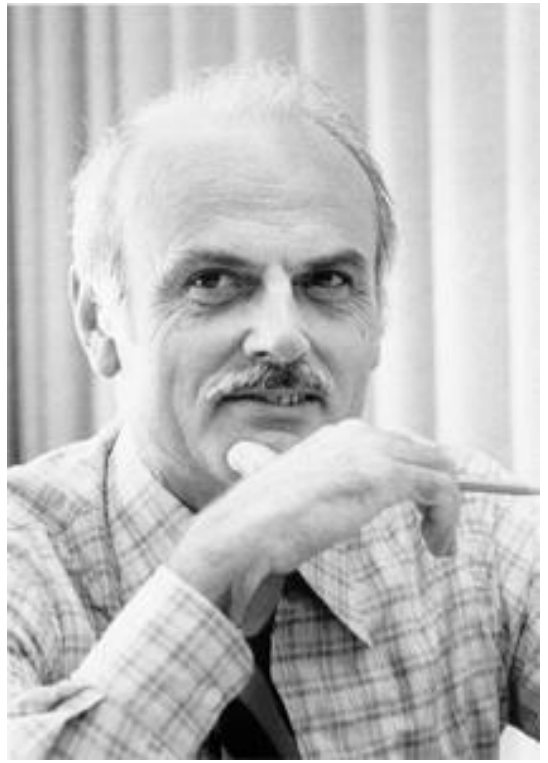
That means a relational database is a collection of two-dimensional tables, also called entities, consisting of:

columns, the so called attributes

rows, the so called tuples, records or instances

Definition II

- Proposed by Dr. Codd in 1970
- The basis for the relational database management system (RDBMS)
- The relational model contains the following components:
  - Collection of objects or relations
  - Set of operations to act on the relations
  - Data integrity for accuracy and consistency



History I [1]

- The term “relational database” was first defined by E. F. Codd at IBM in 1970
- He introduced the term in a research paper and used a later paper to define the meaning of the term “relational”
- The most well-known definition is composed of Codd’s 12 rules [2], however no commercial implementation conforms to all of these rules, since they are simply not always usable in practice

History II [1]

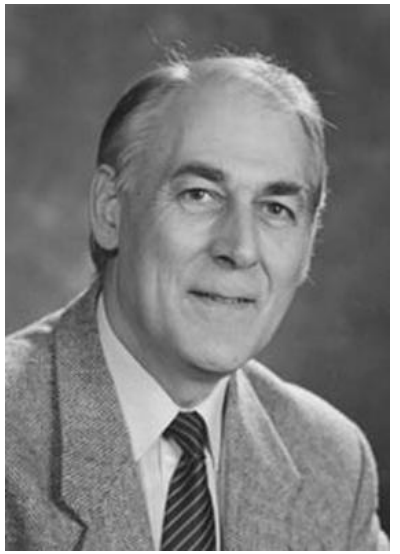
- In 1974 the IBM began developing the first system to manage relational databases, called System R
- The first sold system of this kind was the Multics Relational Data Store in 1976
- It was followed by Oracle, Ingres and IBM BS12
- The first relatively faithful implementations of the relational model by Codd were from the University of Michigan, the MIT and the IBM UK Scientific Centre

History III [1]

- There are two schools of thought regarding these management systems
- The first sees all products that present data as a collection of rows and columns even if they are not strictly based on Codd‘s rules as a relational system
- Nearly all systems nowadays used fall under this school of thought, since they were developed with practicality in mind

History IV [1]

- The second says only systems that implement all of Codd‘s rules or the current understanding as expressed by Christopher J. Date [3] and Hugh Darwen [4] can be seen as relational
- They often differentiate between truly-relational systems and pseudo-relational systems



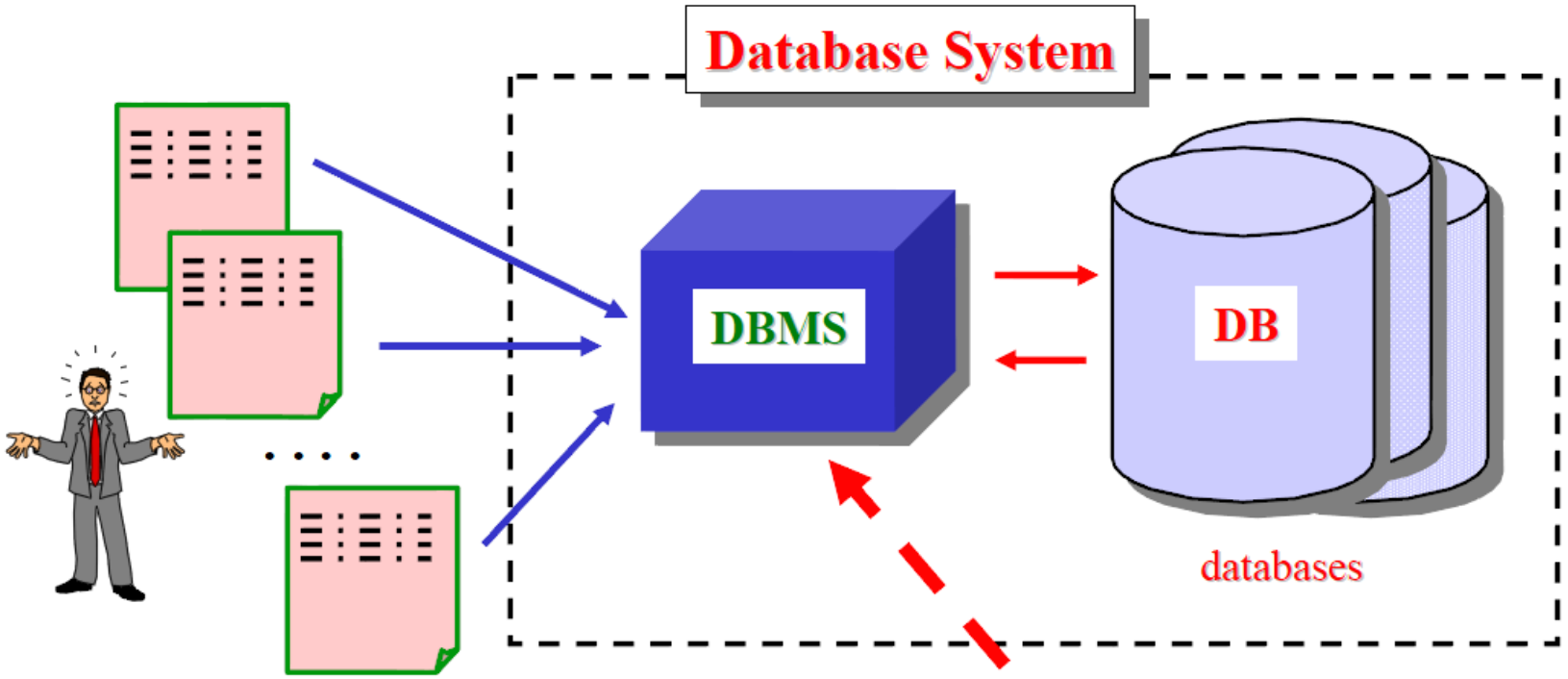
RDBMS

- To interact with our database (the tables) we need a separate program
- These programs are called database management systems
- In combination with a relational database we speak about Relational Database Management Systems (RDBMS)

Relational Database Management System – a database system made up of files with data elements in two-dimensional arrays (rows and columns). This database management system has the capability to recombine data elements to form different relations resulting in a great flexibility of data usage.

~ after Martin, 1976

- A relational database system consists of the following:
  - a relational database (RDB)
  - a relational database management system (RDBMS)
- To interact with this system we have to use the RDBMS and normally we can‘t interact directly with the RDB
- The interaction can stem from:
  - Users (we)
  - Applications (external programs, measurement devices, etc.)

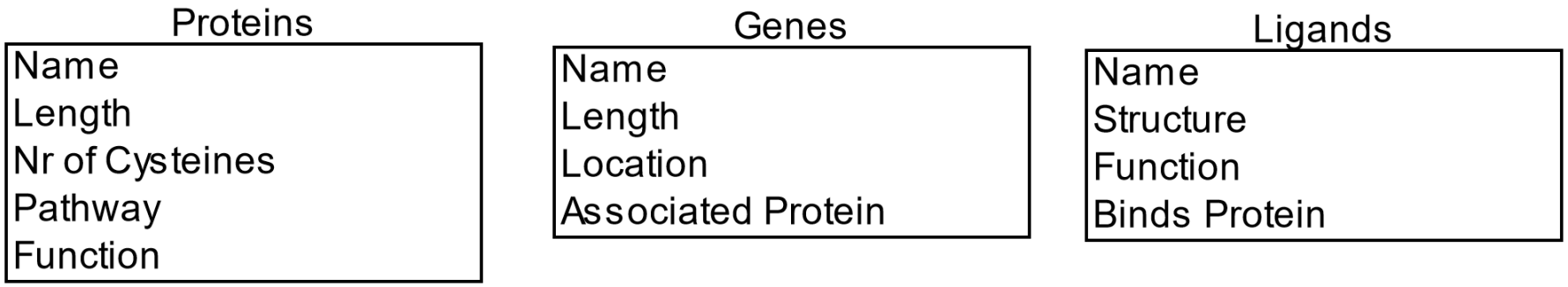

Database System
DBMS
DB
databases
users and
application programmes
DBMS: Database Management System
(application-independent services for
managing data)

Tables

- Are a collection of raw data for a specific topic
- Consist of rows of data (tuples) that are uniquely identified from other rows of data in the same table
- Each row represents or corresponds to a dataset of the topic
- Made of columns or attributes that describe the topic in detail
- Are the logical and perceived data structure, not the physical data structure
- Are abstractions of reality
- We can visualize them by using Class diagrams
- Each column can contain only one value per row, that means we can not save multiple names in a column names for examples

Class Diagrams

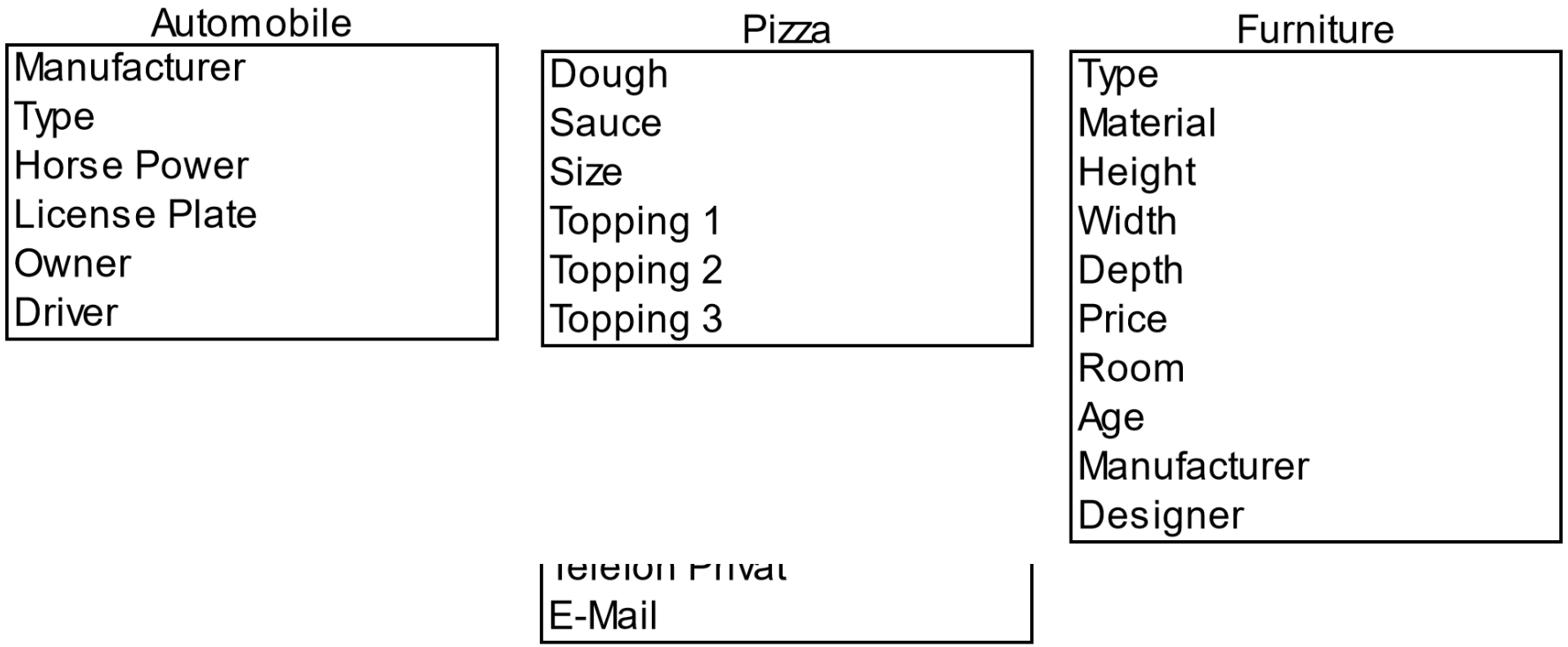
ONE Class Diagram ALWAYS describes ONE table in your database



Exercise 01: Design Tables [5]

- Design tables to represent the following "things" in the given enterprises. For each one draw a class diagram
  - A student at a university.
  - A faculty member at a university.
  - A work of art that is displayed in a gallery or museum.
  - An automobile that is registered with the Motor Vehicle Department.
  - A pizza that is on the menu at a restaurant.
  - Household furniture.

Exercise 01: Solutions



Terminology I

- Each table is composed of rows and columns
- The rows are our datasets and the columns contain specific data for each row
- You can manipulate data in the rows by executing Structured Query Language (SQL) commands as we will learn later

S_CUSTOMER Table (Relation)

Row (Tuple)

ID	NAME	PHONE	SALES_ REP ID
201	Unisports	55-2066101	12
202	Simms Athletics	81-20101	14
203	Delhi Sports	91-10351	14
204	Womansport	1-206-104-0103	11

Column (Attribute)

Terminology II

- Each row of data in a table needs to be uniquely identified by a primary key (PK)
- You can logically relate information from multiple tables using foreign keys (FK), which reference Primary Keys of other tables

ID	NAME	PHONE	SALES_ REP_ID
201	Unisports	55-2066101	12
202	Simms Atheletics	81-20101	14
203	Delhi Sports	91-10351	14
204	Womansport	1-206-104-0103	11

ID	LAST_ NAME	FIRST_ NAME
10	Havel	Marta
11	Magee	Colin
12	Giljum	Henry
14	Nguyen	Mai

Primary Key Foreign Key Primary Key

Primary Key [6]

- A primary key (PK) is a column or a set of columns that uniquely identifies each row in a table
- Each table must have a primary key and a primary key must be unique, that means we can not have duplicate values in the corresponding column
- A PK consisting of multiple columns is called a Composite primary key
- No part of the PK can be null (empty)
- Tips for identifying PKs:
  - Must be a unique value
  - Value in the PK for each tuple or row should never change
  - PK is best auto-generated

Foreign Keys [7]

- A foreign key (FK) is a column or a combination of columns in one table that refers to a primary key in another table
- A FK must match an existing primary key value (or else be null / empty)
- If a FK is part of a primary key, the FK can't be null / empty
- In order for a relation to be established between two tables, they both must contain a common data element
  - e.g. a column that has been defined the same in both tables

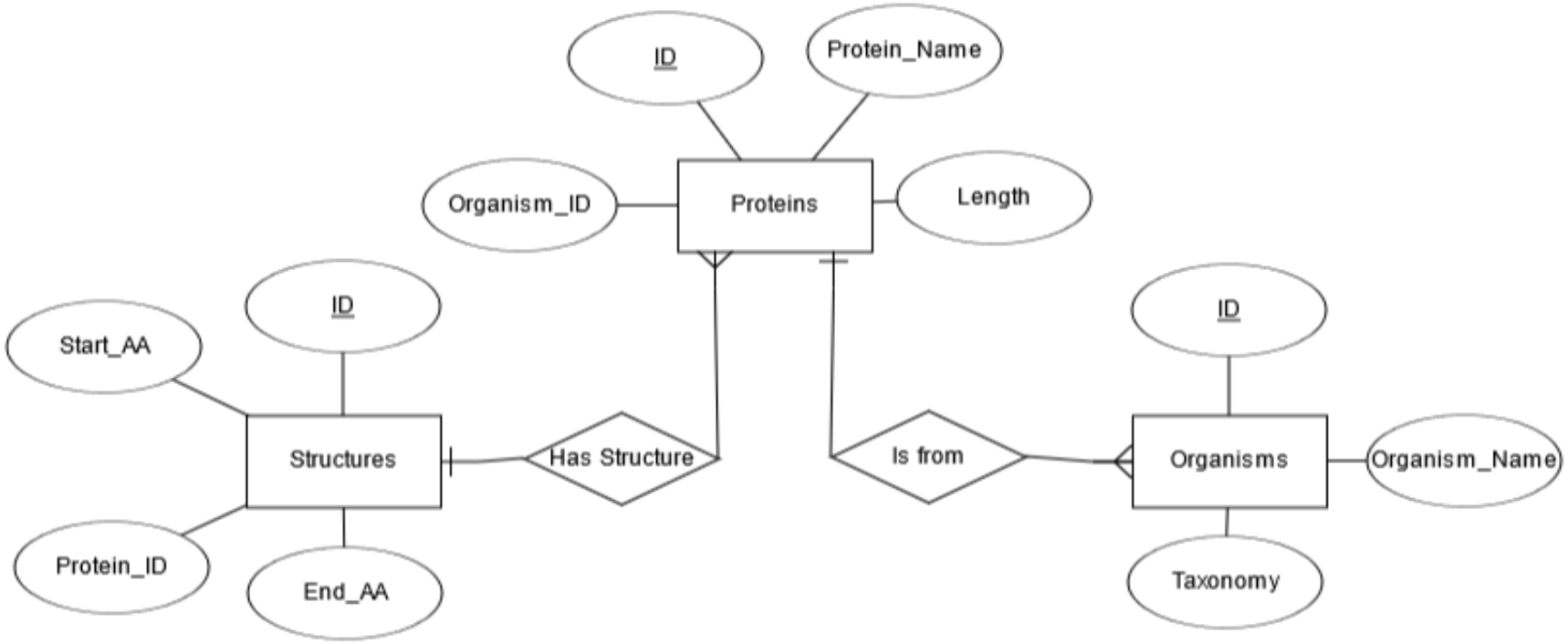
Visualizing I

- There are many ways to visualize / display the schematic structure of a RDB
- The two most common ones are Relational Schemes [8] and ER Diagrams [9]
- Both show the tables with their corresponding columns and allow for visualizing the relations between tables
- You can create both on the following website:
  - https://erdplus.com/standalone

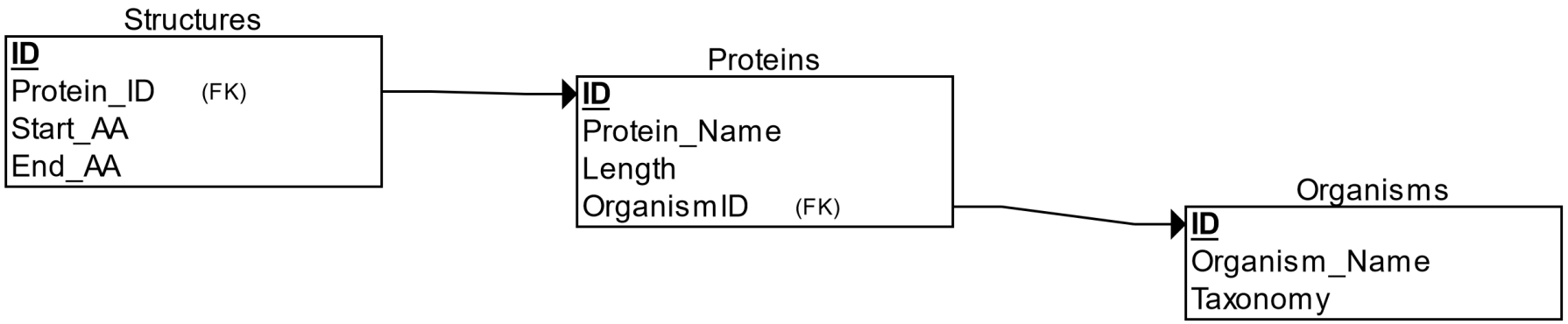
Visualizing II

- I used the following tables for creating the examples
- Table Proteins with columns ID, Protein_Name, Length, Organism ID
- Table Organisms with columns ID, Organism_Name, Taxonomy
- Table Structures with columns ID, Protein_ID, Start_AA, End_AA

Visualizing III – ER-Diagram



Visualizing IV – Relational Scheme



Creating RDBs

Gathering information I [10]

- Gather the requirements and define the objective of your database
- Answering the following questions can help you:
  - What do you need to save in your database?
  - What kind of relations are you interested in?
  - What kind of diagrams, tables do you need for presentations or publications?
  - Do you need to include external applications and their outputs?
- Try to figure out as much details as you can during this first step to make sure that you don‘t need to make too much changes to your database later on
- It may be better to spend one more hour during this step, than needing multiple hours or days later on when you need to add more data / tables

Gathering information II [10]

- Try to figure out nearly everything you want to save in your database, at this step already
- Don‘t think about tables or anything like this during this stage, just write down some keywords that can help you in the design process later on
- The main aim of this step is to create a rough outline of your database, so that you don‘t forget anything during the next steps

Designing Tables I [10]

- Once you have decided on the purpose of the database, gather the data that is needed to be stored in the database
- Divide the data into subject-based tables, that means each table contains information about a separate part of the data you gathered
- For each table determine what kind of columns you need to be able to save the corresponding data sets
- After you determined all columns your table needs, pick one or multiple columns as primary key
- If no columns in your table fulfill the criteria for a primary key (being unique), you need to add another column to contain your primary key

Designing Tables II [10]

- Imagine you want to save information about proteins you‘re experimenting with
- In this case you would need at least the following columns:
  - protein name
  - length
  - function
- Now you need to figure out which column you could use as a primary key
- Unfortunately all three of them are probably not unique
- In such a case we need to add an additional column, e.g. ID, which contains just a simple ongoing number we could use as primary key

Exercise 02: Table design

- Design tables for the following topics:
  - Cell cultures
  - Laboratory animals
  - Isolation experiments
  - Western Blot Results
  - Protein concentration measurements
- Don’t forget to include a primary key for each table

Modelling Relations [10]

- Databases consisting of independent and unrelated tables serve little purpose (consider to use a spreadsheet instead)
- The power of a relational database lies in the relationships that can be defined between tables
- The most crucial aspect in designing a relational database is to identify these relationships among tables
- The types of relationship include:
  - one-to-many
  - many-to-many
  - one-to-one

Modelling Relations – One-to-Many I [10]

- In a "class roster" database, a teacher may teach zero or more classes (a lesson at a specific time point), while a class is taught by one (and only one) teacher
- This kind of relationship is known as *one-to-many*
- One-to-many relationships cannot be represented in a single table (next slide)

Modelling Relations – One-to-Many II [10]

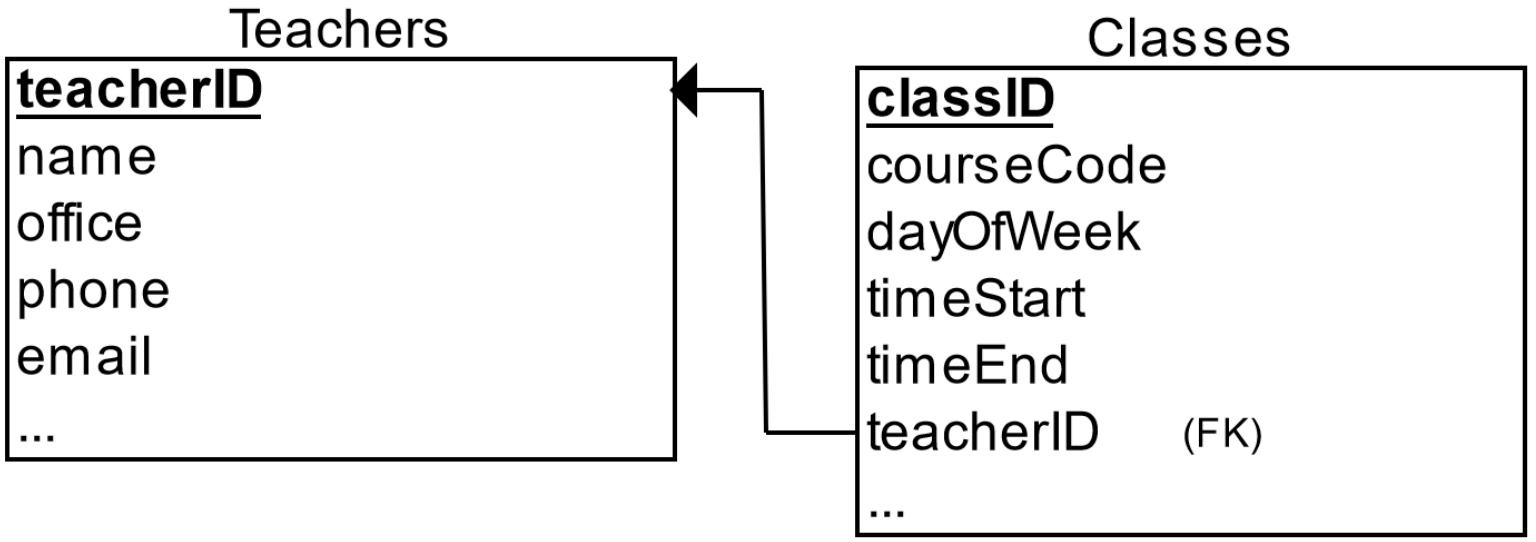
- In a "class roster" database, we may begin with a table called Teachers
  - Stores information about teachers (such as name, office, phone and email)
- To store the classes taught by each teacher, we could create columns called class1, class2, class3 and so on
- However we face different problems:
  1. How many columns do we need?
  2. What happens when a teacher is only responsible for one instead of 5 classes?
- Creating a table like this leads to many empty fields in our table, which could affect the performance of our database significantly

Modelling Relations – One-to-Many III [10]

- On the other hand, if we begin with a table called Classes
  - Stores information about a class (courseCode, dayOfWeek, timeStart and timeEnd)
- We could create additional columns to store information about the (one) teacher (such as name, office, phone and email) that is responsible for the class
- However, since a teacher may teach many classes, its data would be duplicated in many rows in table Classes
- We need to avoid such redundancy at all cost in our databases

Modelling Relations – One-to-Many IV [10]

- That means to support a one-to-many relationship, we need to design two tables:
  - a table Classes to store information about the classes with classID as the primary key
  - a table Teachers to store information about teachers with teacherID as the primary key
- We can then create the one-to-many relationship by storing the primary key of the table Teachers in the table Classes in form of a foreign key:



Modelling Relations – One-to-Many V [10]

- The table teachers in our example is now the parent table of our Relationship, since it contains the primary key we use to model our relation
  - It‘s also the “one”-end of our relation
- The table classes in our example is now the child table of our Relationship, since it contains the foreign key column
  - It’s also the “many”-end of our relation

Exercise 03: One-to-Many

- Create a relational scheme for each of the relations described below:

1. Organisms and Kingdoms of Life
2. Protein sequences and known Protein structures
3. Genes and protein sequences
4. Chromosomes and Genes

Modelling Relations – Many-to-Many I [10]

- In a "product sales" database, a customer's order may contain one or more products
- And a product can appear in many orders
- In a "bookstore" database, a book is written by one or more authors
- While an author may write zero or more books
- This kind of relationship is known as many-to-many

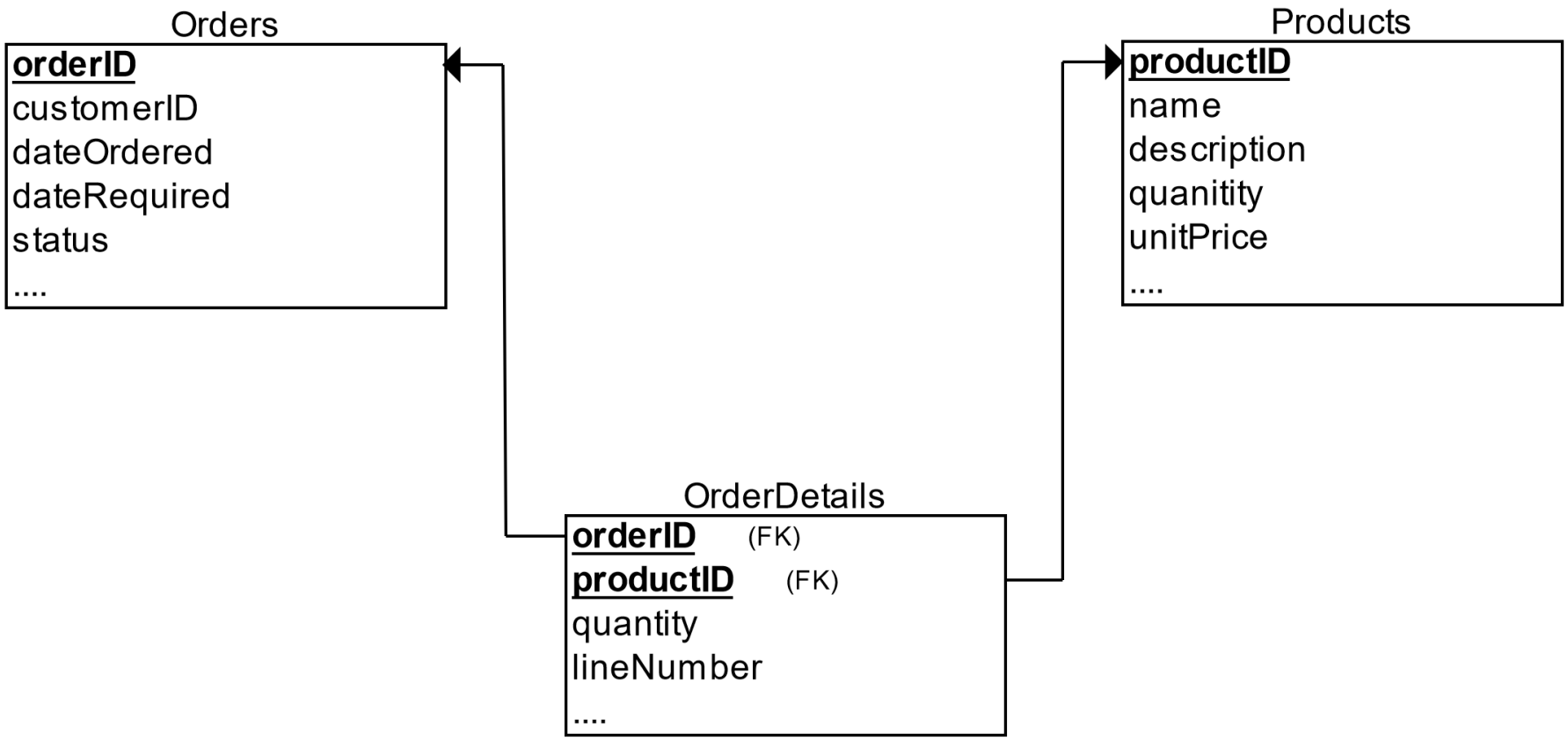
Modelling Relations – Many-to-Many II [10]

- Let's illustrate with a "product sales" database
- We begin with two tables:
  - Products
  - Orders
- The table products contains information about the products (such as name, description and quantityInStock) with productID as its primary key
- The table orders contains customer's orders (customerID, dateOrdered, dateRequired and status)

Modelling Relations – Many-to-Many III [10]

- we cannot store the items ordered inside the Orders table and we also cannot store the order information in the Products table
- To support a many-to-many relationship, we need to create a third table (known as a junction table or relationship table)
  - say OrderDetails (or OrderLines), where each row represents an item of a particular order
- The columns orderID and productID in OrderDetails table are used to reference the Orders and Products tables, hence, they are the foreign keys in the OrderDetails table
- Since the combination of these two foreign key columns is unique throughout our table we can use them as a composite primary key

Modelling Relations – Many-to-Many IV [10]



Modelling Relations – Many-to-Many V [10]

- The many-to-many relationship is, in fact, implemented as two one-to-many relationships, with the introduction of the junction or relationship table
  - An order has many items in OrderDetails. An OrderDetails item belongs to one particular order.
  - A product may appears in many OrderDetails. Each OrderDetails item specified one product.
  - Therefore Orders and Products form a Many-to-Many Relationship

Exercise 04: Many-to-Many

- Create a Relational scheme for each of the three databases described below:

1. A database that collects information about proteins, organic cofactors (e.g. ATP) and inorganic cofactors (e.g. Ca ions)
2. A database that collects information about proteins, organisms and the kingdoms of life
3. A database that collects information about cell lines, chemicals used in experiments and the observation of these experiments

- Note: not all tables in a database form a relation, e.g. kingdoms of life and proteins in (2) aren’t related to each other

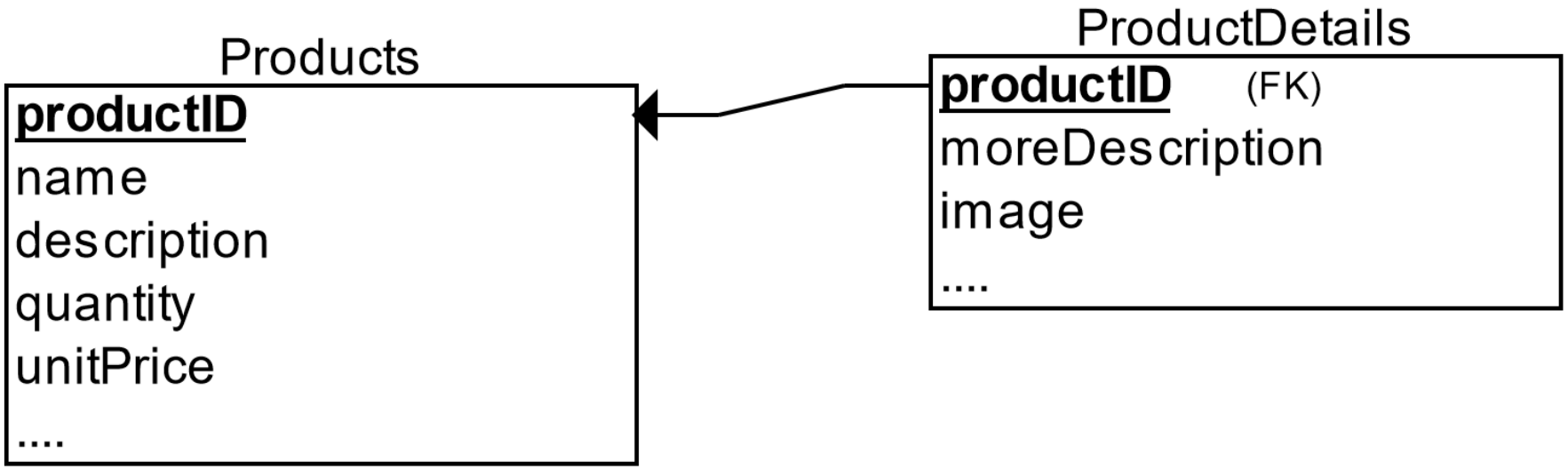
Modelling Relations – One-to-One I [10]

- In a "product sales" database, a product may have optional supplementary information such as image, more Description and comment
- Keeping them inside the Products table results in many empty spaces (in those records without these optional data)
- Furthermore, these large data may degrade the performance of the database

Modelling Relations – One-to-One II [10]

- Instead, we can create another table (say ProductDetails, ProductLines or ProductExtras) to store the optional data
- A record will only be created for those products with optional data
- The two tables, Products and ProductDetails, exhibit a one-to-one relationship
- That is, for every row in the parent table, there is at most one row (possibly zero) in the child table
- The same column productID should be used as the primary key for both tables

Modelling Relations – One-to-One III [10]



Modelling Relations – One-to-One IV [10]

- Some databases limit the number of columns that can be created inside a table
- You could use a one-to-one relationship to split the data into two tables
- One-to-one relationship is also useful for storing certain sensitive data in a secure table, while the non-sensitive ones in the main table

Summary Relations

- If we talk about relations between tables we analyze how one data Point (one entry) of one table depends on the entries in the other table
- So we take one entry in Table 1 (T1) to look at how many entries in T2 it points and vice versa

Table 1 (T1)	Table 2 (T2)	Relationship
Points to one entry in T2	Points to one entry in T1	One-to-One
Points to many entries to T2	Points to one entry in T1	One-to-Many
Points to many entries to T2	Points to many entries to T1	Many-to-Many

Refining the Database I [1]

- For example,
  - adding more columns,
  - create a new table for optional data using one-to-one relationship,
  - split a large table into two smaller tables,
  - others
- Apply the so-called *normalization rules* to check whether your database is structurally correct and optimal

Refining the Database II – Normalization Rules I

- A table is in the first normal form if all columns contain atomic values and an unique primary key is used

PersNr	Last Name	Give Name	Dept Nr	Dept.	Project Nr	Description	Time
1	Lorenz	Sophia	1	HR	2	Sales Promotion	83
2	Hohl	Tatjana	2	Purchase	3	Competitor Analysis	29
3	Willschrein	Theodor	1	HR	1	Customer Survey	140
3	Willschrein	Theodor	1	HR	2	Sales Promotion	92
3	Willschrein	Theodor	1	HR	3	Competitor Analysis	110
4	Richter	Hans-Otto	3	Sales	2	Sales Promotion	67
5	Wiesenland	Brunhilde	2	Purchase	1	Customer Survey	160

Refining the Database III – Normalization Rules II

- A database is in it‘s second normal form if the first normal form is fulfilled and each non-key column is functional fully dependent from the primary key

Projects

Project Nr	Project
2	Sales Promotion
3	Competitor Analysis
1	Customer Survey

Employees

PersNr	Last Name	Given Name	Dept Nr	Dept
1	Lorenz	Sophia	1	HR
2	Hohl	Tatjana	2	Purchase
3	Willschrein	Theodor	1	HR
4	Richter	Hans-Otto	3	Sales
5	Wiesenland	Brunhilde	2	Purchase

Employees in Projects

PersNr	Project Nr	Time
1	2	83
2	3	29
3	1	140
3	2	92
3	3	110
4	2	67
5	1	160

Refining the Database IV – Normalization Rules III

- A database is in it‘s third normal form if the second normal form is fulfilled and no non-key column depends transitively from a key column or no non-key column depends on another non-key column

Employees

PersNr	Last Name	Given Name	Dept Nr
1	Lorenz	Sophia	1
2	Hohl	Tatjana	2
3	Willschrein	Theodor	1
4	Richter	Hans-Otto	3
5	Wiesenland	Brunhilde	2

Departments

Dept Nr	Dept.
1	HR
2	Purchase
3	Sales

General Rules

General Rules I

- When working with and/or creating relational databases you need to keep some general rules in mind to make sure everything runs as smoothly as possible

1. Each Table needs a Primary Key at all time, to uniquely identify each row in it
2. Make sure that the data in your table depends only on the primary key
3. Reduce redundancy to a level of foreign key values
4. Each field can contain only exact one value, even though this value could contain multiple words, e.g. homo sapiens
5. Avoid empty fields as much as possible
6. Model all relations before entering data into your database

Exercises

Exercise 05: Research laboratory

- You need to create a database for a research laboratory that works on analyzing the activity of proteins
- Therefore they need to collect information about:
  - the proteins they analyze
  - the experiments they run
  - tools used during the experiments
  - the results of the experiments in standardized form, that means there are standard results like activity increased or activity decreased

Exercise 06: Plant institute

- A research institute that focusses mainly on research in plants needs a database to store information about the greenhouses they run
- They want to know:
  - which plants are placed in which greenhouse
  - which gardener is responsible for which greenhouse
  - what experiments are run on the plants and in which greenhouse these experiments are done

Exercise 07: Data collection

- Use the table on page 34 of ABI-Exercise.pdf and generate a database out of it
  - make sure it fulfills the general rules for relational databases you learned

Exercise 08: Music collection

- A Music Shop wants a database to collect information about their inventory.
- They sell different kinds of records, e.g. CDs, Vinyl, DVDs, and want to know:
  - which artists were responsible for which albums
  - what kind of records were produced for these albums
- Furthermore, they want to save information about:
  - the music genre, e.g. Rock, Pop,
  - the countries the different artists stem from

Exercise 09: University

- A university wants you to design a database containing information about:
  - students
  - employees
  - courses, e.g. Biology, Chemistry
  - different faculties
  - different departments in these faculties
- For students they need to know which course they visit and some contact information
- Similar they need to know which employee is part of which department and whether they are responsible for certain courses or not

Exercise 10: Zoo

- A Zoo wants you to create a database containing information about:
  - the animals it contains
  - employees working in it
- The Zoo has different zones for animals from the different continents
- All employees are divided into several categories
- Since the Zoo is part of an international breeding program it needs information about:
  - “guest animals”
  - their partner zoos

Exercise 11: Metagenomics

- Your research group focuses on metagenomics studies and needs a database for this purpose.
- The database needs to collect information about:
  - different elemental probes, sorted by type (water, soil, plant material, etc.)
  - the organisms found in these probes
- For each probe several sequencing experiments are executed to determine which different types of nucleic acids (DNA or RNA, single or double-stranded) are present in them
- Store these information also in your database

Definitions

Definitions I

- A **Relational Database** is a collection of tables that are connected with each other
- A **Relational Database Management System (RDBMS)** is a program that allows the user to interact with a relational database
- Each dataset of a table is saved in a **Row**
- Tables consist of multiple **Columns** that contain the data for the different rows of your table
- Each row in a table can be identified with the help of a **Primary Key (PK)**, a column with unique values and no empty fields

Definitions II

- We can refer to rows in other tables by using a **Foreign Key (FK)**, a column that references a primary key column in another table and can contain only values that are present in the corresponding Primary Key column
- A table containing a primary key that is used as foreign key in another table is called **Parent Table**
- A table containing a foreign key is called **Child Table**
- A table containing foreign keys referencing at least two other tables is called **Junction Table** or **Relationship Table**
- A **Field** of a Table refers to one piece of information from a dataset saved in a specific column, you could say the crossing of a column and a row

Literature

1. https://en.wikipedia.org/wiki/Relational_database (Last edited: 26.03.2023, Last visited: 26.04.2023)
2. https://en.wikipedia.org/wiki/Codd%27s_12_rules (Last edited: 31.01.2023, Last visited: 26.04.2023)
3. https://en.wikipedia.org/wiki/Christopher_J._Date (Last edited: 16.10.2022, Last visited: 26.04.2023)
4. https://en.wikipedia.org/wiki/Hugh_Darwen (Last edited: 06.11.2022, Last visited: 26.04.2023)
5. https://home.csulb.edu/~mopkins/cecs323/exercises.shtml (Last edited: 05.09.2021, Last visited: 09.01.2023)
6. https://en.wikipedia.org/wiki/Primary_key (Last edited: 17.04.2023, Last visited: 26.04.2023)
7. https://en.wikipedia.org/wiki/Foreign_key (Last edited: 21.03.2023, Last visited: 26.04.2023)
8. https://en.wikipedia.org/wiki/Database_schema (Last edited: 07.09.2022, Last visited: 26.04.2023)
9. https://en.wikipedia.org/wiki/Entity%E2%80%93relationship_model (Last edited: 17.01.2023, Last visited: 26.04.2023)
10. https://www3.ntu.edu.sg/home/ehchua/programming/sql/Relational_Database_Design.html (Last edited: September 2010, Last visited 07.03.2022)